1. INTRODUCTION

The exponential growth of digital content across documents, images, and videos has transformed how information is created, stored, and consumed. Educational institutions, enterprises, and technical domains increasingly rely on large collections of heterogeneous data for knowledge sharing and decision-making. However, accessing precise information from such vast and multimodal sources remains a significant challenge. Users are often required to manually browse lengthy documents, interpret diagrams, or watch videos to locate specific answers, which is time-consuming and inefficient. This problem highlights the need for intelligent systems capable of understanding and retrieving information directly from diverse user-provided content.

Conventional chatbot systems, though powerful, primarily depend on pretrained large language models with static internal knowledge. These systems are designed to generate responses based on patterns learned during training rather than on the specific documents or materials provided by users at runtime. As a result, they struggle to answer queries grounded in external content and may produce hallucinated or unverifiable responses. This limitation reduces their reliability in domains where factual accuracy and source traceability are critical, such as education, research, enterprise knowledge management, and technical documentation analysis.

To address these shortcomings, this project proposes an Intelligent Question Answering Chatbot based on a Multimodal Retrieval-Augmented Generation (RAG) framework. The core idea of Retrieval-Augmented Generation is to enhance language model responses by retrieving relevant contextual information from an external knowledge base before generating an answer. By integrating retrieval mechanisms with generative capabilities, the system ensures that responses are not only fluent but also factually grounded in user-provided data.

A key innovation of this project is its ability to operate on multimodal data sources. Textual information from native PDF documents is extracted using document parsing techniques. For scanned pages, images, and embedded diagrams, Optical Character Recognition (OCR) is employed to convert visual content into machine-readable text. Videos containing spoken explanations are processed using speech-to-text transcription methods, enabling the system to utilize audio information as part of its knowledge base. Videos without meaningful speech are intelligently filtered using metadata handling to avoid unnecessary processing. This

comprehensive extraction pipeline allows the chatbot to build a unified textual representation from multiple modalities.

The extracted content is then segmented into meaningful semantic chunks to preserve context and improve retrieval precision. These chunks are converted into high-dimensional vector embeddings using pretrained embedding models and stored in a vector database along with contextual metadata such as source references and positional information. This vectorized representation enables efficient similarity-based retrieval when a user submits a query.

During interaction, user queries undergo lightweight normalization and are transformed into embeddings. The system performs a similarity search in the vector database to identify the most relevant content segments related to the query. These retrieved contexts are then passed to a Large Language Model, which generates responses strictly based on the retrieved information. This grounding mechanism significantly reduces hallucinations and ensures that answers are accurate, context-aware, and traceable to their original sources.

By combining multimodal data extraction, vector-based retrieval, and language model generation, the proposed chatbot offers a scalable and reliable solution for conversational information access. It minimizes the need for manual document exploration, enhances knowledge accessibility, and supports informed decision-making across various real-world applications. This approach demonstrates how multimodal Retrieval-Augmented Generation can transform traditional chatbot systems into intelligent assistants capable of reasoning over user-specific knowledge repositories.

2. SUPPORTING LITERATURE

Title	Methodology	Summary
MDKAG: Retrieval-Augmented Educational QA Powered by a Multimodal Disciplinary Knowledge Graph	OCR for documents, text–image extraction from slides, ASR-based transcription for videos, graph-based retrieval	The study shows that integrating multimodal knowledge graphs with RAG improves answer accuracy and reduces hallucinations compared to traditional RAG systems. Experimental results demonstrate better retrieval relevance and reliable question answering across different educational subjects, making the approach scalable and effective for educational applications.
Multimodal retrieval-augmented generation framework for visually rich knowledge in the architecture domain	CLIP, VisualBERT, and Doubao embedding models for multimodal retrieval; Qwen2.5-VL for table parsing; Qwen3 MLLM for multimodal answer generation	Proposes a multimodal RAG framework that unifies text, images, and tables for accurate, traceable knowledge retrieval and generation in architectural design tasks
Knowledge graph-extended retrieval augmented generation for question answering	KG-based multi-hop retrieval, and similarity-based triple selection to generate explainable answers without model training	training-free question answering framework that integrates Large Language Models (LLMs) with Knowledge Graphs (KGs) to improve accuracy and explainability.

Table 1.1 Literature Review

2.2 Literature Review summary

Recent research in Retrieval-Augmented Generation (RAG) shows a clear shift from text-only retrieval toward structured and multimodal knowledge integration to reduce hallucinations and improve answer reliability in question answering systems. The three papers reviewed collectively demonstrate how RAG systems evolve when they incorporate knowledge graphs, multimodal document understanding, and reasoning-aware retrieval strategies.

The first work, MDKAG: Retrieval-Augmented Educational QA Powered by a Multimodal Disciplinary Knowledge Graph by Xu Zhao, Guozhong Wang, and Yufei Lu, addresses limitations of traditional RAG in educational environments. The authors observe that educational content exists in textbooks, slides, and lecture videos, but standard RAG treats this data as flat text chunks. They propose constructing a multimodal disciplinary knowledge graph from OCR-processed documents, PPT slides, and video transcripts. Retrieval is performed through entity relationships in the graph rather than pure embedding similarity, and a post-generation verification mechanism checks semantic similarity and entity coverage to reduce hallucinations. This work highlights that knowledge organization and verification are as important as retrieval.

The second work, Multimodal Retrieval-Augmented Generation framework for visually rich knowledge in the architecture domain by Xianchuan Meng and Ziyu Tong, focuses on challenges in domains where information is highly visual, such as architecture. The authors argue that converting tables, diagrams, and drawings into plain text embeddings destroys structural meaning. Their M-RAG framework decomposes tables into both semantic text and rendered images, embeds text and images into a shared semantic space, and uses a Multimodal LLM to interpret retrieved visual content before answer generation. This paper emphasizes that multimodal embedding alignment and visual reasoning are essential for accurate retrieval in visually rich documents.

The third paper, Knowledge graph-extended retrieval augmented generation for question answering by Jasper Linders and Jakub M. Tomczak, explores how Knowledge Graphs can be combined with RAG without requiring training. Building on earlier systems like KAPING, the authors introduce a question decomposition module that uses Chain-of-Thought prompting to split complex queries into sub-questions. This improves multi-hop triple retrieval and produces an explicit reasoning chain, making answers more explainable. The

study demonstrates that reasoning-aware retrieval significantly improves performance for complex queries.

Across these works, a common theme emerges: traditional RAG fails when knowledge is multimodal, structured, or requires multi-step reasoning. The solutions proposed in these papers extend RAG in three complementary directions:

- Knowledge Graph integration for structured, entity-aware retrieval and explainability (KG-RAG, MDKAG)
- Multimodal data handling for tables, diagrams, and images (M-RAG)
- Question decomposition and reasoning chains to improve multi-hop retrieval (KG-RAG)
- Answer verification mechanisms to reduce hallucinations (MDKAG)

Together, these studies form a strong foundation for building advanced RAG systems capable of operating on heterogeneous user-provided data while maintaining answer accuracy, contextual relevance, and traceability.

2.3 Findings and Proposals

The study of existing chatbot and Retrieval-Augmented Generation systems shows that traditional chatbots relying solely on pretrained language models are unreliable when answering questions from user-provided documents, as they often generate hallucinated or unverifiable responses. Even conventional RAG systems, although improving factual grounding through retrieval, are largely limited to text-only processing and struggle with real-world data that includes scanned PDFs, diagrams, tables, images, and videos. Additionally, retrieval based purely on vector similarity can return contextually irrelevant content if semantic segmentation and metadata are not properly preserved. These observations indicate that reliable document-centric question answering requires multimodal data extraction, semantic indexing, accurate retrieval, and response generation strictly grounded in retrieved content.

Based on these findings, this project proposes an intelligent multimodal Retrieval-Augmented Generation chatbot for answering questions from user-uploaded documents and videos. The system extracts text from native PDFs using PyMuPDF, applies OCR for scanned documents and diagrams, and converts spoken content in videos into text using Whisper, ensuring all information is transformed into a unified textual form. The processed content is semantically

segmented, embedded using the all-MiniLM-L6-v2 model, and stored in a Chroma vector database for efficient similarity-based retrieval. When a user submits a query, relevant content is retrieved and passed to the LLaMA 3 language model, which generates responses strictly based on this context. This approach reduces hallucinations, improves answer accuracy, and provides scalable, reliable conversational access to heterogeneous data sources.

3. Problem Definition

With the rapid growth of digital content, large volumes of information are now stored in the form of PDF documents, scanned files, images, diagrams, presentations, and videos. In educational institutions, enterprises, and technical domains, users frequently need to extract specific information from such document collections. However, locating precise answers within lengthy and heterogeneous files is a time-consuming and inefficient process that often requires manual reading, searching, and interpretation.

Although modern chatbots powered by Large Language Models appear capable of answering questions conversationally, they suffer from a fundamental limitation: they rely on pretrained internal knowledge rather than the user's actual documents. As a result, when users ask questions related to their own files, these chatbots tend to produce hallucinated, generic, or unverifiable responses. Even basic Retrieval-Augmented Generation systems improve this only partially, as they primarily work with plain text and fail to handle information embedded in scanned pages, diagrams, tables, and videos.

In real-world scenarios, important knowledge is frequently present in non-textual formats. Diagrams in technical manuals, scanned academic notes, tables in reports, and spoken explanations in videos contain critical information that traditional text-based retrieval systems cannot process effectively. This creates a gap between how information is stored and how intelligent systems attempt to retrieve it.

Therefore, there is a strong need for a system that can:

- Extract meaningful information from multimodal sources such as PDFs, images, and videos,
- Organize this information in a retrievable form,
- Retrieve the most relevant context for a user's query, and
- Generate answers that are strictly grounded in the user-provided content, ensuring accuracy and traceability.

4. Identify and Analyse the tools/Models

PyMuPDF for Text Extraction

PyMuPDF is a Python binding for the MuPDF engine, designed for fast and precise reading of digitally created (machine-readable) PDFs. It is not an AI model; it is a high-performance PDF parsing library that exposes the internal structure of a PDF page—text blocks, lines, words, coordinates, fonts, and metadata.

In a multimodal RAG system, the quality of retrieval depends heavily on how accurately document text is extracted before embedding. Many PDFs (reports, notes, manuals) are digitally generated and already contain structured text. If this text is extracted naively (e.g., plain copy), the reading order, columns, headers, and positional context are lost. PyMuPDF preserves this structure, which directly improves:

- Semantic chunking quality
- Embedding relevance
- Retrieval accuracy from the vector database

PyMuPDF serves as the foundation of the document understanding pipeline for native PDFs. By preserving layout, structure, and reading order, it enables high-quality semantic chunking and embedding, which are critical for accurate retrieval and reliable answer generation in the multimodal RAG chatbot.

Optical Character Recognition (OCR) for Scanned Documents

Optical Character Recognition (OCR) is a computer vision + pattern recognition pipeline that converts text present inside images (scanned PDFs, photos, diagrams, screenshots) into machine-readable digital text. In your multimodal RAG chatbot, OCR is essential because a large portion of real-world documents are not digitally typed PDFs. They are:

- Scanned class notes
- Printed documents saved as images
- Diagrams with labels
- Tables captured as images
- Old reports and handwritten annotations (sometimes)

Without OCR, all this content is invisible to embeddings, vector search, and the LLM. OCR is what unlocks this hidden knowledge and brings it into the same retrieval pipeline as normal text.

How an OCR System Works Internally

A typical OCR engine (e.g., Tesseract OCR) follows a multi-stage pipeline:

1. Image Preprocessing
    - Convert to grayscale
    - Noise removal (denoising)
    - Binarization (black–white thresholding)
    - Skew correction (straightening tilted scans)
    - Contrast enhancement
2. Text Detection & Segmentation
    - Detect text regions in the image
    - Segment into blocks → lines → words → characters
    - Draw bounding boxes around detected text areas
3. Character Recognition
    - Pattern matching or deep learning model recognizes characters
    - Converts pixel shapes into letters and numbers
4. Post-processing (Language Correction
    - Dictionary-based correction
    - Language model correction (fixes “teh” → “the”)
    - Reconstruct proper sentences from recognized words

Why OCR is Critical for RAG Pipeline

In a multimodal RAG system, embeddings can only be created from text. If a page is scanned:

- Embedding model sees nothing
- Vector DB stores nothing
- LLM cannot answer from that content

OCR converts this non-readable content into retrievable knowledge.

Whisper for Speech-to-Text Transcription

Whisper is a transformer-based Automatic Speech Recognition (ASR) model designed to convert spoken audio into accurate text across multiple languages and accents. It is trained on a very large, diverse audio dataset and is robust to background noise, varied speaking styles, and domain vocabulary. In your multimodal RAG chatbot, Whisper is the component that converts knowledge hidden inside videos (lectures, explanations, demos) into searchable text that can be embedded, stored, and retrieved just like document content.

Why Speech-to-Text is Essential in our System

A large amount of real knowledge today exists in:

- Lecture recordings
- Tutorial videos
- Meeting recordings
- Explanatory demonstrations

Without speech-to-text, this knowledge is invisible to embeddings and vector search. The LLM cannot access it. Whisper transforms this audio knowledge into text so it becomes part of the same retrieval pipeline as PDFs and OCR output.

Architecture of Whisper

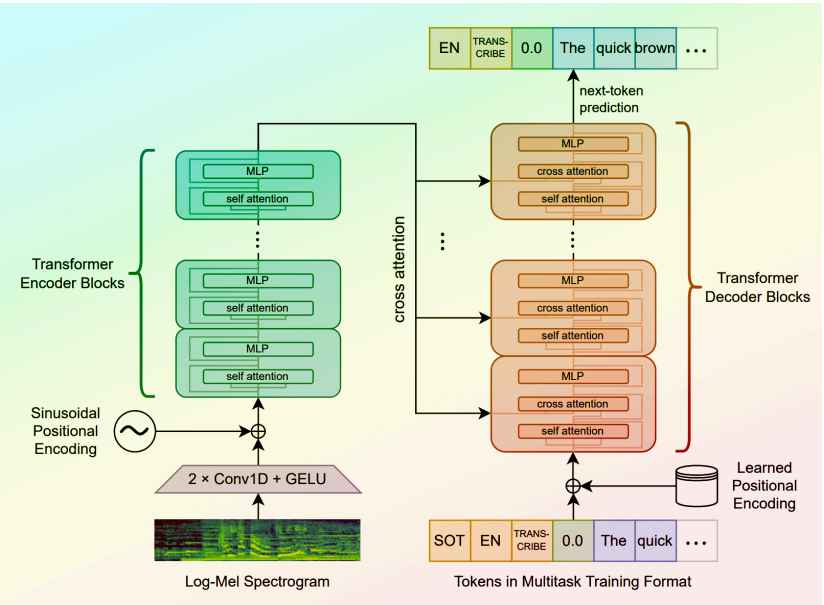

Figure 1.1: Architecture of Whisper

Whisper follows an encoder–decoder transformer architecture:

1. Audio Preprocessing
    - Input audio is converted into a log-Mel spectrogram (a visual representation of sound frequencies over time).
    - This turns audio into a format similar to an image that transformers can process.
2. Encoder
    - The encoder processes the spectrogram and learns patterns of speech, tone, pitch, and phonetics.
    - It produces a compressed representation of the spoken content.
3. Decoder
    - The decoder predicts the corresponding text tokens step by step.
    - It uses language understanding to correct mistakes and produce grammatically meaningful sentences.
4. Timestamp Alignment
    - Whisper can produce timestamps for each sentence/segment.
    - This allows mapping text back to exact video time locations.

all-MiniLM-L6-v2 for Semantic Text Embeddings

The all-MiniLM-L6-v2 model is a sentence embedding model from the Sentence Transformers family, specifically designed to convert text into dense numerical vector representations that capture semantic meaning rather than surface-level word similarity. In this project, it plays a central role by transforming all extracted textual content whether from PDFs, OCR output, or video transcripts into embeddings that can be efficiently stored, searched, and compared within a vector database.

Unlike traditional keyword-based or lexical search methods, all-MiniLM-L6-v2 enables semantic retrieval, meaning that conceptually similar text segments can be retrieved even if they do not share exact keywords. This capability is essential for Retrieval-Augmented Generation systems, where the quality of retrieval directly determines the accuracy of the generated responses.

What Problem Embedding Models Solve

In document-centric QA systems, raw text cannot be directly compared in a meaningful way. Consider the following:

- “What is retrieval-augmented generation?”
- “Explain how RAG improves LLM accuracy”

Although these sentences use different words, they express the same intent. Keyword matching would treat them as different, but an embedding model maps them to nearby points in vector space, enabling accurate retrieval.

The all-MiniLM-L6-v2 model solves this problem by encoding semantic meaning, context, and intent into a fixed-length vector, making similarity computation mathematically feasible.

Internal Architecture and Design

all-MiniLM-L6-v2 is based on the MiniLM architecture, which is a compressed transformer model derived from larger language models using knowledge distillation. Key architectural characteristics include:

- 6 transformer layers, making it lightweight and fast
- 384-dimensional output embeddings
- Transformer-based self-attention mechanism
- Optimized for sentence-level semantic similarity

The model is trained using contrastive learning, where semantically similar sentence pairs are pulled closer in vector space, while unrelated pairs are pushed farther apart. This training objective is what allows the model to capture meaning rather than syntax.

1. Input Text Processing
    - Each semantic chunk (from PDFs, OCR text, or video transcripts) is tokenized into subword units using a transformer tokenizer.
2. Contextual Encoding
    - Tokens pass through multiple transformer layers where self-attention captures relationships between words, phrases, and sentence-level context.
3. Sentence Representation

- Token-level representations are pooled (mean pooling) to produce a single dense vector representing the entire chunk.

4. Vector Normalization
    - The output vector is normalized, enabling efficient cosine similarity computation during retrieval.

LLaMA 3 8B for Answer Generation

LLaMA 3 (8B parameters) is a state-of-the-art Large Language Model developed by Meta and is used in this project as the core generative component responsible for producing final user-facing answers. Unlike earlier chatbot systems that rely solely on pretrained internal knowledge, LLaMA 3 is integrated into a Retrieval-Augmented Generation (RAG) pipeline, where its output is explicitly grounded in externally retrieved information. This design choice ensures that the model functions not as a standalone conversational agent but as a controlled reasoning engine that generates responses strictly based on the contextual evidence supplied to it.

LLaMA 3 is built on a decoder-only transformer architecture, which has become the dominant paradigm for modern generative language models. The transformer design enables the model to capture long-range dependencies in text using self-attention mechanisms rather than sequential recurrence. The 8-billion-parameter variant represents a balanced configuration that provides strong language understanding and generation capabilities while remaining computationally feasible for deployment in applied systems. This balance is particularly important in document-centric applications, where inference efficiency and response latency are critical factors.

Internally, the model processes input text by first converting tokens into dense vector representations through an embedding layer. These embeddings pass through multiple stacked transformer blocks, each consisting of multi-head self-attention layers and feed-forward neural networks. The self-attention mechanism allows the model to dynamically focus on different parts of the input context, assigning higher importance to relevant tokens while generating each output token. This capability is especially important in RAG-based

systems, where the prompt includes retrieved document excerpts that must be accurately referenced during answer generation.

In the context of this project, LLaMA 3 does not operate on raw user queries alone. Instead, the system constructs a structured prompt that includes the user's question along with the most relevant contextual chunks retrieved from the vector database. These chunks originate from PDFs, OCR-processed scanned documents, and Whisper-transcribed video content. By supplying this retrieved context directly to the model, the system constrains LLaMA 3 to base its responses on verifiable external information, significantly reducing hallucination and improving factual correctness.

A key strength of LLaMA 3 lies in its strong instruction-following behavior and contextual reasoning ability. When prompted appropriately, the model can synthesize information across multiple retrieved chunks, identify relationships between concepts, and produce coherent, well-structured answers. This is particularly useful when user questions require summarization, explanation, or comparison rather than direct fact lookup. The model's capacity to integrate information across multiple context passages allows it to function as a reasoning layer on top of the retrieval system.

Another important aspect of LLaMA 3 is its scalability and controllability within a RAG architecture. Since the model does not require fine-tuning for each new document collection, it can be reused across different datasets simply by updating the retrieval corpus. This aligns well with the project's objective of supporting user-specific documents, where the knowledge base may change frequently. The separation between retrieval and generation ensures that updates to the document set do not require retraining the language model itself.

From a reliability perspective, the use of LLaMA 3 in a grounded generation setting directly addresses one of the most critical weaknesses of large language models: hallucination. In traditional chatbot systems, the model may generate plausible-sounding but incorrect answers when information is missing or ambiguous. In contrast, this project restricts the model's input to retrieved context, effectively limiting its output space to information that exists within the user-provided data. As a result, answers are not only more accurate but also traceable to their original sources.

The choice of the 8B parameter variant is also significant. Larger models may offer marginal gains in fluency, but they come with increased computational cost and deployment complexity. The 8B model provides a practical compromise by delivering high-quality language generation while remaining suitable for real-time inference in an applied system. This makes it particularly appropriate for academic and enterprise use cases, where resource constraints must be considered alongside performance.

In summary, LLaMA 3 (8B) serves as the final reasoning and response generation engine in the proposed multimodal RAG chatbot. By operating within a retrieval-augmented framework, the model combines the expressive power of large language models with the reliability of document-grounded knowledge access. Its transformer-based architecture, strong contextual reasoning abilities, and efficient parameter scale make it well-suited for generating accurate, coherent, and context-aware answers from heterogeneous user-provided data. This integration transforms LLaMA 3 from a generic conversational model into a dependable component of a document-centric question answering system.

Architecture of LLaMA 3

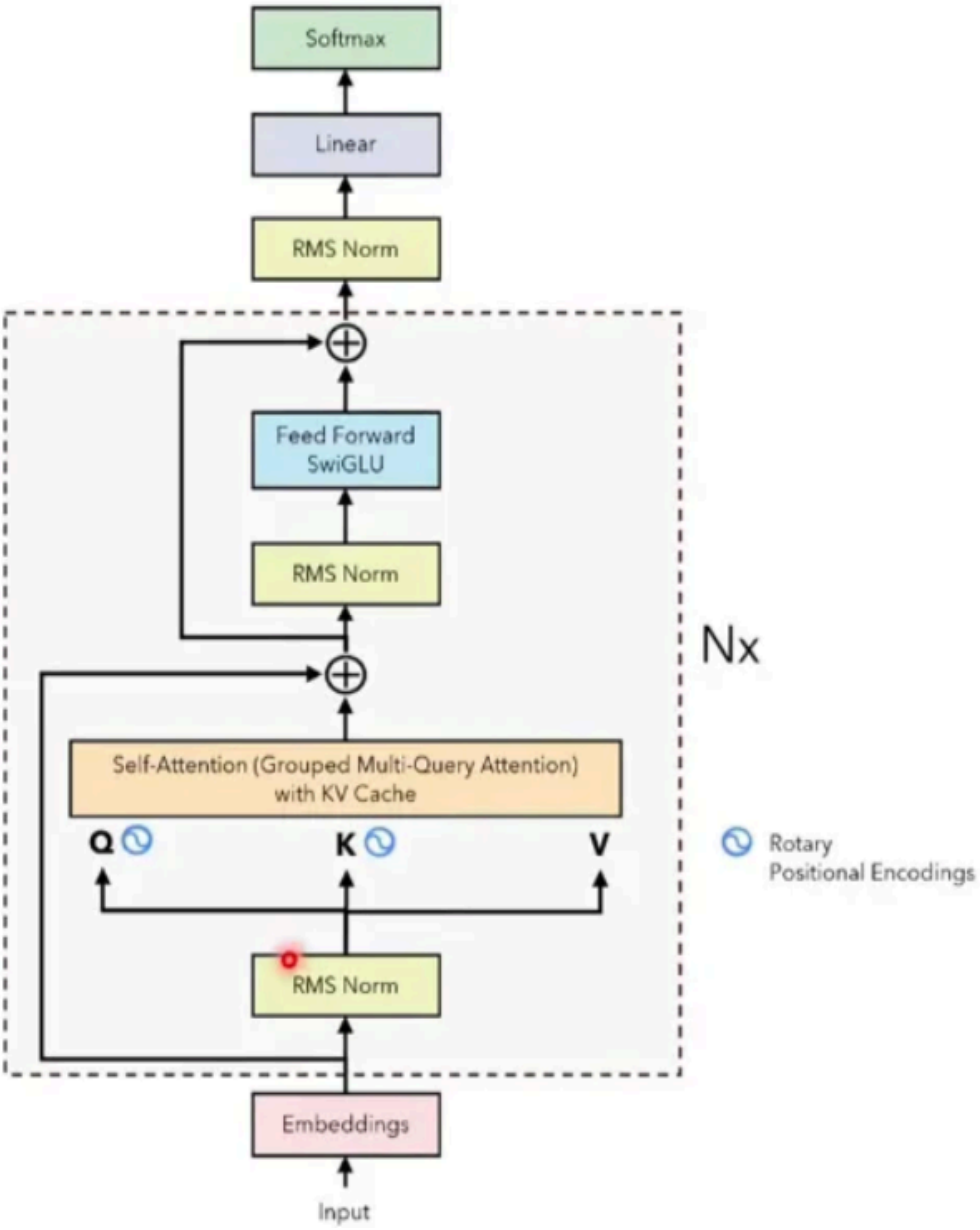

Figure 1.2 Architecture of LLaMA 3

LLaMA 3 follows a decoder-only Transformer architecture, optimized for efficient large-scale text generation. The model consists of an input embedding layer followed by N repeated transformer blocks, and finally a projection layer that produces token probabilities.

Each component in the architecture is designed to improve training stability, inference efficiency, and contextual reasoning.

- Input Embeddings

The architecture begins with token embeddings, where each input token (generated using a tokenizer) is mapped to a dense vector representation. These embeddings encode semantic meaning and serve as the initial numerical representation of the input text.
Unlike traditional RNN-based models, LLaMA 3 processes the entire input sequence in parallel using these embeddings.

- RMS Normalization (Pre-Norm Design)

Before entering both the attention and feed-forward sublayers, RMSNorm (Root Mean Square Normalization) is applied.
Purpose of RMSNorm:

➔ Stabilizes training for very deep transformer networks
➔ Reduces computational overhead compared to LayerNorm
➔ Improves numerical stability during inference

LLaMA 3 uses a pre-norm architecture, meaning normalization is applied *before* each major operation rather than after. This helps prevent gradient instability as depth increases.

- Self-Attention with Grouped Multi-Query Attention (GQA)

The core reasoning component of each transformer block is the Self-Attention layer, implemented using Grouped Multi-Query Attention with KV Cache.
Self-Attention
Self-attention allows each token to attend to all previous tokens in the sequence, enabling the model to capture long-range dependencies and contextual relationships.
Each token produces:

➔ Query (Q) – what the token is looking for
➔ Key (K) – what the token offers
➔ Value (V) – the information passed forward

Attention weights are computed between Q and K, then applied to V.

Grouped Multi-Query Attention (GQA)

Instead of using separate Key and Value matrices for every attention head (as in standard Multi-Head Attention), LLaMA 3 shares K and V across groups of heads.

Why GQA is used:

- ➔ Significantly reduces memory usage
- ➔ Improves inference speed
- ➔ Maintains high attention quality

This design is especially important for large models like LLaMA 3, enabling scalable deployment without sacrificing performance.

KV Cache

During text generation, previously computed Key and Value tensors are stored in a KV cache. This avoids recomputation for past tokens and allows fast autoregressive decoding.

- Rotary Positional Embeddings (RoPE)

Instead of absolute positional embeddings, LLaMA 3 uses Rotary Positional Embeddings applied directly to the Query and Key vectors.

Advantages of RoPE:

- ➔ Encodes relative positional information naturally
- ➔ Improves performance on long contexts
- ➔ Enables extrapolation to longer sequences than seen during training

RoPE allows the model to understand order and distance between tokens without adding explicit position vectors.

- Residual Connections

After the self-attention output, a residual (skip) connection adds the original input back to the output.

- Feed-Forward Network with SwiGLU

Following attention, the output passes through a Feed-Forward Neural Network (FFN) implemented using SwiGLU activation.

SwiGLU (Swish-Gated Linear Unit)

SwiGLU replaces traditional ReLU or GELU activations.

Benefits:

- ➔ Improves expressiveness
- ➔ Enables gated information flow
- ➔ Enhances language modeling performance
- ➔ The FFN expands the hidden dimension, applies SwiGLU activation, and then projects it back to the original size.

- Second RMSNorm and Residual Connection

Another RMSNorm is applied before the FFN, followed by a residual connection after it.

This completes one transformer block.

- Repeated Transformer Blocks (N×)

The dashed box labeled N× indicates that this entire block—attention + FFN—is repeated multiple times.

- Final RMSNorm, Linear Layer, and Softmax

After passing through all transformer blocks:

- ➔ Final RMSNorm stabilizes output representations
- ➔ Linear layer projects hidden states to vocabulary size
- ➔ Softmax converts logits into probability distributions over tokens

The token with the highest probability is selected during generation.

5. Project Pipeline

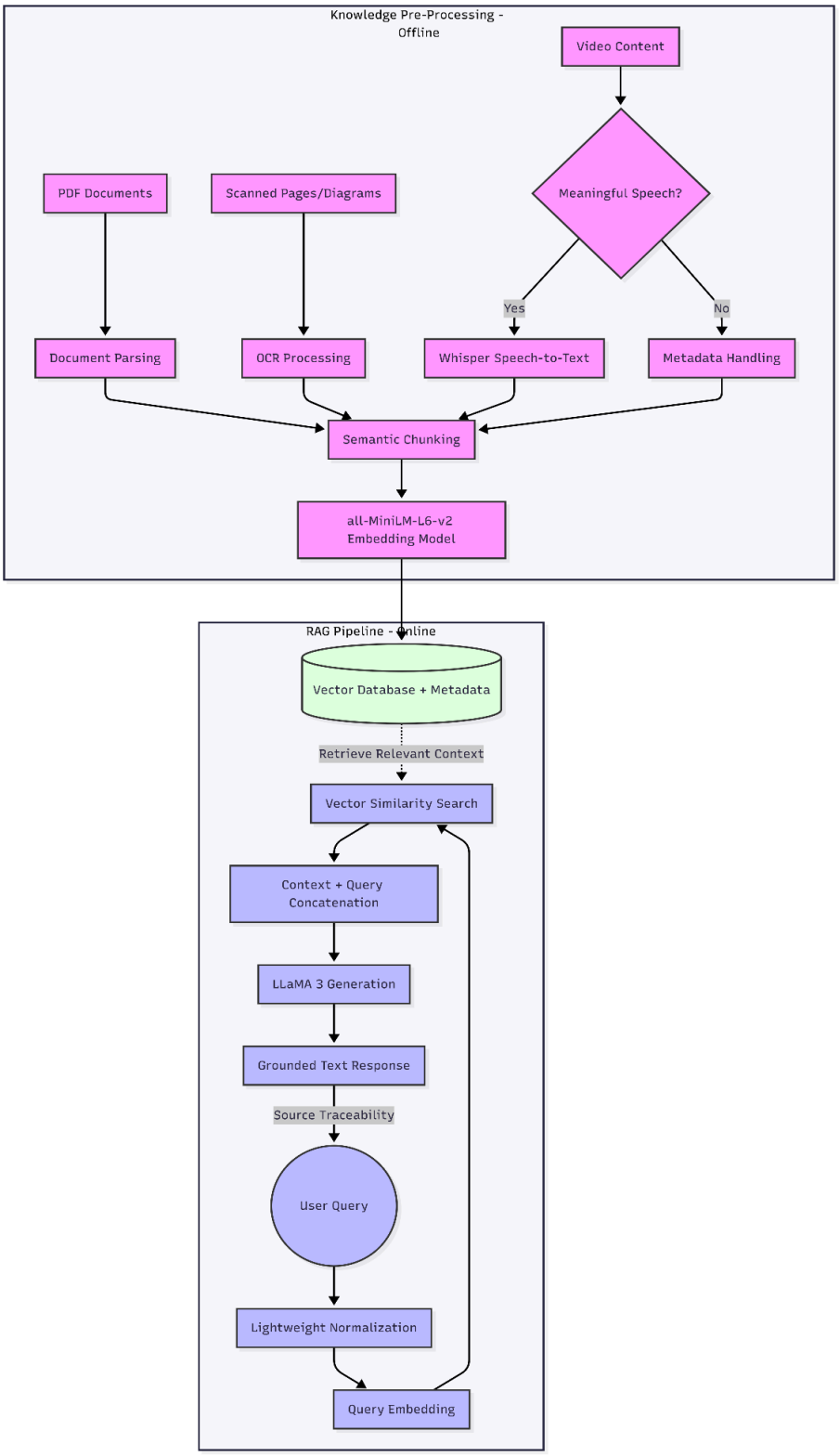

Figure 1.3: Project pipeline

1. User Interaction and Data Ingestion:

The pipeline begins with user interaction, where the user uploads documents such as PDF files, images, or videos and submits natural language questions through the chatbot interface. The uploaded files act as the dynamic knowledge source for the system, enabling personalized and document-centric question answering.

2. Multimodal Content Processing

To handle heterogeneous input formats, the system converts all modalities into a unified textual representation:

- PDF Processing:

Native PDFs are parsed to extract textual content page-wise, while scanned PDFs are processed using Optical Character Recognition (OCR).

- Image Processing:

Images containing textual information, diagrams, or figures are processed using OCR techniques to extract embedded text.

- Video Processing:

For videos containing spoken content, the audio stream is extracted and converted into text using automatic speech recognition (ASR). Videos without informative speech are handled using metadata and skipped during transcription.

3. Text Preprocessing and Structure-Aware Chunking

The extracted text is cleaned and segmented into smaller, meaningful units known as chunks. Instead of naïve fixed-length chunking, the system adopts a structure-aware chunking strategy, where document elements such as titles, section headings, content blocks, and footers are identified and labeled using metadata. This improves retrieval accuracy, especially for structural queries such as titles or section-specific questions.

4. Vectorization and Knowledge Indexing

Each text chunk is transformed into a dense vector representation using a pretrained sentence embedding model. These embeddings capture the semantic meaning of the text. The vectors, along with associated metadata (page number, document source, content type), are stored in a vector database (ChromaDB). This indexed vector store forms the searchable knowledge base of the chatbot.

5. Query Processing and Retrieval

When a user submits a query, the question is preprocessed and converted into an embedding using the same embedding model. The vector database performs a similarity search to retrieve the top-k most relevant chunks based on semantic similarity and metadata filters. This retrieval step ensures that only contextually relevant information is selected for answer generation.

6. Context Augmentation and Answer Generation

The retrieved text chunks are aggregated and combined with the user query to form an augmented context. This context is passed to a Large Language Model (LLM), which generates a response conditioned strictly on the retrieved evidence. By grounding the generation process in retrieved content, the system minimizes hallucinations and improves factual consistency.

7. Response Delivery and Explainability

The generated answer is presented to the user along with optional references such as source document names or page numbers. This enhances transparency and allows users to verify the origin of the information. The chatbot supports conversational interaction, enabling follow-up questions within the same context.

8. System Evaluation and Validation

The system is evaluated using retrieval and answer quality metrics such as precision at k, semantic similarity, exact match, and qualitative user assessment. These evaluations measure the effectiveness of retrieval accuracy, contextual relevance, and response reliability.

6. REFERENCES

- XuZhao, Guozhong Wang*andYufei Lu MDKAG:Retrieval-Augmented Educational QA Powered by a Multimodal Disciplinary Knowledge Graph, *Appl. Sci.* **2025**, *15*(16), 9095; **https://doi.org/10.3390/app15169095**
- Meng, X., Tong, Z. Multimodal retrieval-augmented generation framework for visually rich knowledge in the architecture domain. *ARIN* 4, 22 (2025). https://doi.org/10.1007/s44223-025-00102-6
- Jasper Linders1 · Jakub M. Tomczak, Knowledge graph-extended retrieval augmented generation for question answering, Applied Intelligence (2025) 55:1102 https://doi.org/10.1007/s10489-025-06885-5